

How ARCANA AI was built

A student uploads their notes. ARCANA reads every word, turns the material into a 2D adventure game with lessons, mini-games, fights and boss quizzes, and then adapts to how that student learns. This page explains what was built, which tools were used, how they fit together and why each choice was made.

Live app: arcana-ai-16xh.onrender.com · Code: github.com/adarshmen0n/arcana-learning-game

~10,000

lines of code

13

browser script files

26

server modules

7

automated test suites

3

outside Python
packages

CONTENTS

- | | |
|-----------------------------------|----------------------------|
| 1 What ARCANA AI is | 2 Who did what |
| 3 The big picture | 4 Tools and why |
| 5 How a game is made | 6 Inside the game |
| 7 How it teaches and adapts | 8 Rewards and motivation |
| 9 Accounts and security | 10 Speed, phones and scale |
| 11 How it was built, step by step | 12 Problems we solved |
| 13 Testing and deployment | 14 Limits and next steps |
| 15 Questions a mentor may ask | 16 Glossary |

01 What ARCANA AI is

ARCANA AI is a web app and installable phone app that turns any study material into a playable learning game. The main idea: students learn better when they are playing, so the app hides real teaching inside a game they actually want to finish.

- **Input:** a PDF, Word file, PowerPoint, text, web page or even a photo of notes.
- **Output:** a side-scrolling adventure. Each chapter covers one part of the material: mentors teach it, knowledge tablets summarise it, three mini-games and a fight test it, and a boss quiz checks it.
- **After playing:** the app tracks what the student knows per topic, adjusts difficulty, draws a roadmap of what to learn next, and offers an AI assistant (Arc Search) for questions.

The rule behind the design is **teaching first**: at least about 60% of every chapter is teaching (mentors and tablets) and the rest is practice. Every question is written only from what the lessons taught, and is checked against the student's own text.

02 Who did what

This project was built through **AI-assisted development**. Adarsh Menon was the product owner: he decided what the app should be, tested it, rejected designs he did not like, and gave every requirement. The code was written by **Claude Code**, an AI coding assistant made by Anthropic, which works directly in the project folder: it reads files, writes code, runs tests, opens the app in a browser to check it, and deploys it.

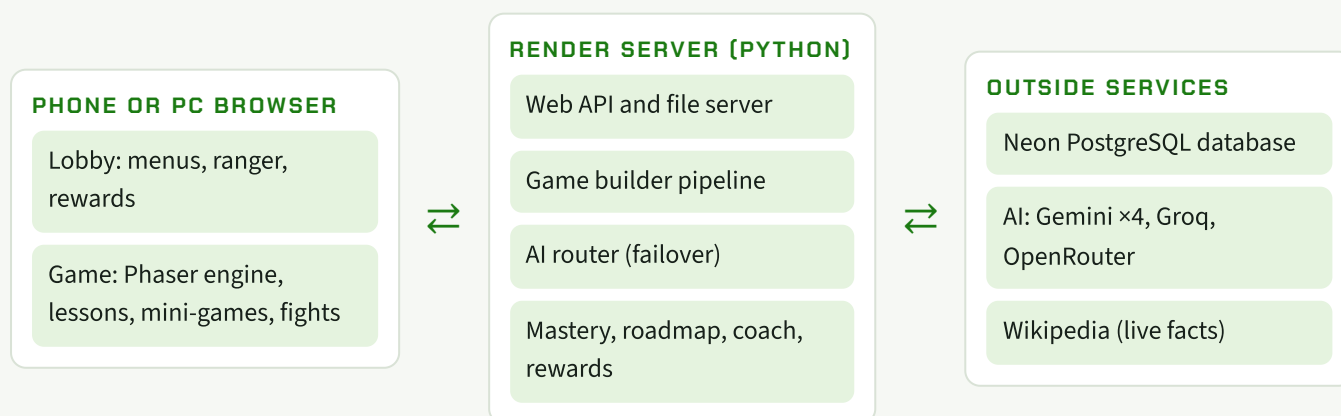
HOW TO DESCRIBE IT TO A MENTOR

"I designed and directed the product: the idea, the features, the look, the priorities and the testing on my phone. I used an AI coding agent to write and test the code under my direction. I understand how the parts fit together, and this page is my record of every decision."

Examples of decisions that came from Adarsh and changed the build: rejecting a glass-style dashboard and a business-style dashboard in favour of a real game lobby, asking for 60% teaching instead of mostly questions, demanding mature characters instead of childlike ones, removing the daily limit, and requiring every chapter to have three mini-games and its own scenery.

03 The big picture

The app has three layers: what runs in the student's browser, the server that does the heavy work, and outside services the server talks to.



- **Browser:** draws the lobby and the game, handles controls and sound, and sends answers back to the server.
- **Server:** accounts, saving progress, reading uploaded files, building games with AI, and the learning model.
- **Database:** stores accounts, games, answers, mastery and rewards so nothing is lost when the server restarts.
- **AI services:** write the lessons and questions. Several are used so the app keeps working when one is busy.

04 Tools used, and why each one

TOOL	WHAT IT DOES HERE	WHY IT WAS CHOSEN
HTML, CSS, JavaScript	All screens: login, lobby, menus, game overlays	Runs in every browser with no install. Plain JavaScript (no React or similar) keeps the app small and fast on phones and needs no build step.
Phaser 3	The 2D game engine: drawing, camera, animation, particles, scenes	A mature, free engine made for browser games. It uses the graphics card (WebGL), so scrolling worlds and fights stay smooth.
Canvas drawing in code	Every character, enemy, world and effect is drawn by code, not image files	No artwork to buy or license, tiny downloads, and every character can change colour (suits and trims) instantly.
Web Audio	Lobby music and sound effects, generated live	No audio files to download; music can change mood per world.
PWA (manifest + service worker)	Lets the site install like an app on a phone home screen	An app-store app would need separate Android and iPhone builds. A PWA is one codebase that installs from the browser.
Python (standard library server)	The web server and API	Python is great for text processing and AI work. Using its built-in web server means almost no dependencies to break.
pypdf	Reads text from PDFs	Pure Python and reliable; works on the free server without extra system software.
psycopg	Talks to the PostgreSQL database	The standard Python driver for PostgreSQL.
anthropic	Optional connection to Claude	If a Claude key is added, it gives the highest-quality lessons and web research.

TOOL	WHAT IT DOES HERE	WHY IT WAS CHOSEN
SQLite / PostgreSQL (Neon)	The database (SQLite on a laptop, PostgreSQL online)	SQLite needs no setup for development. Online, Render's free plan erases its disk on restart, so data lives in Neon's free PostgreSQL instead.
Gemini, Groq, OpenRouter	The AI that writes lessons, questions and answers	All have free tiers. Each free key has a limit, so the app uses several keys and switches between them automatically.
BM25 search (written in the project)	Finds the most relevant passages of the student's text	Used to check every question against the original text, and to give Arc Search the student's own notes. Written in-house so it needs no extra service.
Wikipedia API	Free live facts	Fills gaps in thin material and keeps Arc Search up to date, at no cost.
Render	Hosts the live app	Free web hosting that redeploys automatically whenever new code is pushed to GitHub.
Git and GitHub	Version history and backup of all code	Every change is a saved commit (32 so far), so any version can be restored, and Render deploys from it.

05 How a game is made from an upload

This is the core of the project: the **game builder pipeline** (`server/pipeline.py`). When a student presses *Forge my game*, these steps run in order:

- 1 Wait for a slot.** At most 3 games are built at once. Others wait in a line and see "2 games ahead of you". This protects the free AI limits when many students upload together.
- 2 Read the file.** `ingest.py` extracts the text from PDF, DOCX, PPTX, TXT, Markdown, HTML or CSV. Scanned PDFs and photos have no text layer, so they are sent to Gemini's vision model to be read like a person would read them.
- 3 Check the shared cache.** The text is fingerprinted (SHA-256). If anyone already built a game from exactly the same material, it is reused instantly with no AI calls. Useful when a whole class uploads the same chapter.
- 4 Split into chapters.** The full text is cut, in order, into parts of about 220 words (at least 2, at most 20). Every word belongs to exactly one part, so nothing is skipped and longer material makes a longer game.
- 5 Plan.** The AI reads everything and returns a plan: a title, the list of concepts, one chapter per part, a mood for each chapter (science, nature, history and so on), boss names and mentor names. The AI must answer in a fixed format (JSON with a schema), which the server checks.
- 6 Fill gaps.** Concepts the AI marks as thin or possibly out of date are looked up on Wikipedia, with sources kept.

- 7 **Write each chapter.** Up to 3 chapters are written at the same time. Inside a chapter, the lessons (4 mentors) and the 2 knowledge tablets are written in parallel, then two sets of questions are written in parallel, using only what the lessons taught.
- 8 **Coverage check.** The server compares the lessons with the original part word by word. If less than 92% was taught, the AI writes an extra "More to know" mentor for the missing parts.
- 9 **Quality checks on questions.** A second AI pass re-answers every question and removes doubtful ones; repeated questions are removed; and a grounding check (BM25) removes any question that the student's own text does not support.
- 10 **Assemble the chapter.** Lessons, tablets, 3 mini-games, two fights, a match, a mission, a trial and a mini-boss are placed in order, with a world, weather and boss chosen so each chapter looks different.
- 11 **Validate and save.** The finished game is checked for structure, saved to the database, and only then unlocked for the student.

IF THE AI IS BUSY

Free AI services sometimes run out of quota. The builder spreads calls across all keys, waits up to 10 minutes for a busy service, and if all are still busy it builds a quick version (fill-in-the-blank questions made without AI) so the student can always play. Pressing Rebuild later adds full AI lessons.

How the AI router works

`server/llm.py` treats every AI service as interchangeable. Each request goes to the least busy key of the preferred service. If it fails (limit reached, server overloaded, model retired), the router marks that key as "cooling down" and tries the next one automatically. This is called **failover**, and it is the reason the app works on free AI plans.

06 Inside the game

The order of a chapter

Every chapter follows the same rhythm, so students learn the pattern and focus on the content:

1. Mentor 1 introduces the topic, a **knowledge tablet** summarises key points, then an **ambush fight**.
2. Mentor 2 teaches the core ideas, followed by **mini-game 1**.
3. Tablet 2 and Mentor 3 go deeper, then a **match** against a rival and **mini-game 2**.
4. Mentor 4 recaps the chapter, then **mini-game 3**.
5. A **mission** (put steps in order or match terms), a **trial** quiz, an **elite fight**, and the **mini-boss**.

The four mini-games

Each chapter plays three of them, rotating. All ask questions from the student's material and show a short "why" explanation after a right answer.

- **Maze Run** (Pac-Man style): reach the terminal with the right answer while sentries chase you.
- **Snake Trail**: steer into the right answer to grow; wrong answers shrink you.
- **Hill Climb Rally**: drive through the gate with the right answer; hit a ramp fast to jump over wrong ones, and watch your fuel.
- **Invaders** (Space Invaders style): shoot the block with the right answer while dodging fire.

Fights

Fights are real-time with buttons (keyboard on PC, on-screen buttons on phones): a 4-hit combo, kicks, a charged guard breaker, launcher and air combos, throws, sweeps, dodges, perfect parries and four powers. Every enemy carries a "knowledge shard": a fact from the student's own material that is shown when it is defeated. Ambushes come in 2 waves; elite fights in 3 waves plus a gold champion. Difficulty follows the student's mastery, so fights stay hard but winnable.

Every chapter looks different

There are 6 hand-coded worlds (crystal cave, ancient forest, ember citadel, desert canyon, aurora peaks, neon grid). No world appears twice in a row, and when worlds repeat they come at another time of day or weather (dusk with embers, night with rain and lightning, snow and others). Each chapter also gets its own enemy line-up, armour colours, champion and boss.

Characters

Characters are built like puppets: separate head, torso, arms and legs drawn in code and joined at the joints. Animations (strikes, kicks, running, breathing) are keyframes that rotate those joints with easing curves, the same idea used in professional 2D games.

07 How it teaches and adapts

- **Mastery per topic** (`mastery.py`): every answer updates a score from 0 to 1 for that concept. Right answers raise it (more for harder questions and streaks), wrong answers lower it, and using a hint counts for less. Topics are then labelled weak, learning or strong.
- **Adaptive difficulty**: the student's accuracy, pace and mistakes set their difficulty level (hearts, boss strength, rival skill, fight toughness).
- **Roadmap**: a world map of every chapter showing done, mastered, needs review and locked, with what to study next.
- **AI coach** (`coach.py`): looks at habits like guessing, speed and forgetting, and gives a short study plan.

- **Remedial practice** (`remedial.py`): builds extra practice for weak topics from the same uploaded text.
- **Arc Search** (`arc.py`): a built-in AI assistant. It checks Wikipedia for facts that change over time, cites sources, uses the student's own notes, and is locked during quizzes and fights so it cannot be used to cheat.

08 Rewards and motivation

- **XP and levels** from real study (right answers and chapters cleared), with a level-up celebration.
- **Daily login streak** with a flame in the top bar; day 7 pays a jackpot.
- **Three daily challenges** (for example "answer 10 questions correctly"), counted from real answers.
- **A 20-tier season pass** that unlocks shards, ranger suits and glow trims.
- **Armory**: knowledge shards from fights buy permanent upgrades; powers unlock with level.
- **Leaderboard**, which any student can hide from.

These are built in `server/rewards.py` and `server/hero.py`. Every reward is earned by studying, so the game loop always pushes back toward learning.

09 Accounts and security

- **Passwords are never stored.** Only a script hash with a random salt is kept, so even the database owner cannot read them.
- **Sessions** use a secure, HttpOnly cookie that lasts 30 days, so students stay logged in.
- **Privacy between students:** every game, answer and score belongs to one account and the server checks ownership on every request.
- **Rate limits** stop password guessing and abuse.
- **Safe display:** all text from uploads or AI is escaped before it is shown, so a malicious file cannot inject code.
- **Keys stay on the server** in a private `.env` file that is never uploaded to GitHub; the browser never sees an AI key.
- **AI safety:** uploaded material is passed to the AI as data, with instructions to never follow commands found inside it.
- **Account tools:** password reset, download my data, delete my account, terms and privacy pages, and a support inbox.

10 Speed, phones and many users

Phones

- Phones automatically use a lighter graphics mode: no full-screen bloom glow, fewer background layers, fewer particles, lighter character textures and no scanline overlay.

- The game is capped at 60 frames per second (many phones have 120 Hz screens, which doubled the work).
- Only stations near the screen are drawn; the lobby's animated stage pauses when it is off-screen or hidden.
- The fight display updates about 15 times a second and only when a value changes.
- Menus stay upright (portrait) with the ranger on top; the game turns sideways (landscape) for play, with see-through touch buttons laid out like a mobile game.

Many users at once

- Game files are compressed (gzip, about 70% smaller) and cached by the browser, so returning visits load faster.
- The database uses a pool of 8 connections so many requests run side by side.
- Game building waits in a line, reuses identical games, and spreads AI calls across 4 Gemini keys plus Groq and OpenRouter.
- A load test of 40 simultaneous users sending 400 requests passed with no errors.

11 How it was built, step by step

1. First version

Upload to game pipeline, the game engine, accounts, tests, and the first deployment to Render.

2. Permanent storage

Moved the online database to Neon PostgreSQL because the free server forgets its disk on restart.

3. Teaching first

After playing for half an hour, Adarsh found too many questions and too little teaching. Lessons became 4 mentors plus 2 knowledge tablets per chapter (about 60% teaching), and questions are now written only from what was taught.

4. Combat and Arc Search

Added real-time fights with many techniques, and the Arc Search assistant with live sources.

5. Design rounds

Two dashboard redesigns (glass style, then business style) were rejected as not feeling like a game. The final design is an upgraded neon game lobby with a living stage.

6. Full coverage

Every word of the upload is now taught, with a coverage check that adds extra lessons for anything missed.

7. Big upgrade

Mature characters and dangerous enemies, wave fights with champions, Snake and Hill Climb mini-games, the full rewards system, and mobile layout fixes.

8. Real-world fixes

Fixed an upload crash, removed the daily limit, let Gemini read scanned PDFs, handled phone cloud files, and fixed phone orientation.

9. Speed and scale

Phone performance mode, build queue, shared game cache, quick-mode backup, compression, database pool, and spreading AI calls over several keys (a build went from about 4½ to about 2¾ minutes for the same file).

10. Final structure

Real chapters (at least 2), three mini-games in every chapter, a different world and fight per chapter, and the game unlocks only when all chapters are ready.

12 Problems we hit, and how they were solved

PROBLEM	CAUSE	FIX
Free AI kept failing	Each free key has per-minute and daily limits; some models get overloaded	An AI router with automatic failover, several keys, waiting and retrying, and a quick-mode backup
Data disappeared	Render's free disk is wiped on restart	Moved to Neon PostgreSQL
Uploads crashed on some files	A bug when a short file split into exactly two parts	Fixed the splitting code and added tests for many file sizes
"Needs a Claude key" on PDFs	Slide PDFs were treated as scanned, and only Claude could read scans	Use the PDF's text when it has real text; send real scans to Gemini
Word file showed 0 KB on a phone	The file was still in Google Drive, not downloaded	A clear message telling the student to download it first
Game lagged on phones	Full-screen glow effects, too many layers, 120 Hz screens, constant screen updates	Lighter phone mode, 60 fps cap, drawing only what is visible
Mini-games missing	The question checker sometimes removed all of a mini-game's questions	Mini-games borrow other questions from the chapter, so they are never dropped
Only one chapter for small files	Chapters were cut every 450 words	Chapters of about 220 words, at least 2 per game
Slow game building	Every AI call went to the same key and steps ran one by one	Calls spread across keys, independent steps run in parallel

13 Testing and deployment

There are 7 automated test suites (about 160 checks) that run in seconds without using real AI credits. They use stand-in AI services to test:

- **File reading** (`test_ingest.py`): PDFs, Word, PowerPoint and text.
- **AI failover** (`test_ai_path.py`): rate limits, bad answers, rejected keys and quick-mode backup.
- **Game building** (`test_generation.py`): full coverage and chapter splitting.
- **The platform** (`test_platform.py`): accounts, privacy between students, mastery, roadmap and limits. It also runs against a real PostgreSQL database.
- **Rewards** (`test_rewards.py`), **mastery maths** (`test_mastery.py`) and **scale** (`test_scale.py`): the build queue, game cache, compression and 40 simultaneous users.

Every change was also checked in a real browser at PC, Windows-scaled and phone sizes before release.

Deployment flow: code is committed with Git, pushed to GitHub, and Render rebuilds and restarts the live app automatically within about a minute. A deploy script also copies the private keys from `.env` into Render's settings without printing them.

14 Honest limits and next steps

- **Free server sleeps** after 15 minutes without visitors, so the first visit takes 20 to 60 seconds. Render's paid plan (about \$7 a month) removes this and gives more power for a big launch.
- **Free AI limits:** on very busy days building is slower. More free keys (extra Gemini accounts, Cerebras, Mistral) or one paid key would remove most waiting.
- **Build time:** about 2 to 5 minutes for a small PDF, depending on how many chapters it has and how busy the AI services are.
- **Not yet tested on many real phones;** phone sizes were tested in a desktop browser.
- **Possible next steps:** teacher dashboards for classes, multiplayer quiz battles, more worlds, offline play, and native app-store versions.

15 Questions a mentor may ask

How do you stop the AI from making things up?

Four layers: the AI is told to use only the uploaded material; questions are written only from the lessons; a second AI pass re-answers every question; and a search-based grounding check removes any question the original text does not support.

How do you know all of the material is taught?

The text is split so every word belongs to one chapter, and a coverage check compares the lessons against the original. Below 92% coverage, an extra lesson is written for the missing parts.

Why not one big AI call for the whole game?

Free AI services have size and time limits, and one failure would ruin the whole game. Small, checked steps can be retried, run in parallel and moved to another AI service.

Why Python and plain JavaScript instead of popular frameworks?

Fewer dependencies mean fewer things to break, faster loading on phones, and a free server can run it easily. The whole server needs only 3 outside packages.

How does it handle many students at once?

A build queue, a shared cache of finished games, compressed and cached files, a database connection pool, and AI calls spread over several keys. A load test with 40 simultaneous users passed.

What happens if the AI is completely down?

The app still builds a playable quick version from the text, and Arc Search answers directly from the notes or Wikipedia.

How is student data protected?

Hashed passwords, secure cookies, strict ownership checks on every request, escaping of all displayed text, rate limits, and AI keys kept only on the server.

16 Glossary

AI model / LLM

A large language model, the kind of AI that reads and writes text (Gemini, Claude, Llama).

API

A way for programs to talk to each other. The browser talks to the server's API; the server talks to the AI services' APIs.

API key

A secret password that lets the app use an AI service.

Rate limit / quota

The maximum number of AI requests a key may make per minute or per day.

Failover

Switching automatically to a backup service when one fails.

Pipeline

A fixed sequence of steps that turns input (notes) into output (a game).

JSON schema

A strict template the AI's answer must match, so the server can check it.

BM25

A classic search-ranking formula that scores how well a passage matches a question.

Grounding

Checking that every generated fact can be found in the source material.

Phaser / WebGL

Phaser is the game engine; WebGL is the browser feature that lets it use the graphics card.

PWA

Progressive Web App: a website that can be installed and opened like a phone app.

Service worker

A small background script that lets the app open quickly and work while the server wakes up.

Database / PostgreSQL

Where accounts, games and progress are stored permanently.

Deployment

Putting a new version of the app online.

Git / commit

Git records every change to the code; a commit is one saved change with a message.

Cache

Keeping a copy of something so it does not need to be fetched or made again.

ARCANA AI, created by Adarsh Menon. Built with Claude Code (Anthropic). Facts on this page reflect the project as of 9 October 2026.